

Vectorized Semantic Search Engine for Wikipedia

Andrew Xie, Martine Ferrençy

Abstract

Wikipedia's search function relies primarily on exact text matching, which often fails to retrieve relevant articles when queries do not match article titles or content exactly. This limitation makes it difficult for users to find information without knowing precise terminology. We propose a vectorized semantic search engine for Wikipedia that addresses this problem by representing queries and articles as TF-IDF vectors and ranking results using cosine similarity. Our system encodes the semantic meaning of text, enabling retrieval based on conceptual similarity rather than exact keyword matches. We evaluate our approach on the KeySearchWiki dataset, which provides natural language queries with relevance judgments. Our results demonstrate that TF-IDF vectorization provides a functional baseline for semantic search on Wikipedia, while also revealing limitations in handling complex semantic relationships. This work establishes a foundation for future improvements using neural embedding methods and highlights the challenges of semantic search on large-scale knowledge bases.

1 Introduction

1.1 Motivation

The search function on Wikipedia can be frustrating for finding the article a user wants. This is due to the fact that Wikipedia search relies mostly on exact text matches in articles and does not take into account the underlying meaning of a search query. Without knowing the exact title of an article, it will often be missing from search results even if the user's query is very close in meaning. For example, a user searching for "effect of rising temperatures on glaciers" might not retrieve the "Ice-albedo feedback" article, despite its relevance, simply because the query terms do not appear prominently in the article title or opening text.

While Wikipedia search does include features such as stem matching, fuzzy search, and wildcards,

they are either unintuitive to use or inadequate in addressing the underlying issue with text-based search. Stem matching helps with morphological variations (e.g., "running" vs. "run"), but fails to capture synonyms or related concepts. Fuzzy search can handle typos, but requires users to know which terms to apply it to. Wildcards demand that users understand query syntax and predict where variations might occur. None of these features address the fundamental problem: Wikipedia's search engine does not understand the semantic relationship between query concepts and article content.

This limitation is particularly problematic given Wikipedia's role as one of the most widely consulted information resources on the internet. Users come to Wikipedia with information needs expressed in natural language, often without knowledge of the precise terminology used in relevant articles. A search system that can bridge this semantic gap would significantly improve the user experience and information accessibility.

1.2 Proposed Solution

We propose an alternative search engine for Wikipedia using vectorized semantic search. The system converts a query to a vector and compares it to vectors created from the entire knowledge base to find the most similar articles. These vectors encode the semantic meaning of a given query or article through TF-IDF (Term Frequency-Inverse Document Frequency) weighting, which captures both the importance of terms within individual documents and the whole corpus.

Our approach consists of four main stages: 1. text preprocessing, including tokenization, stop-word removal, and stemming; 2. TF-IDF vectorization of both queries and Wikipedia articles; 3. cosine similarity calculation between query vectors and all article vectors; and 4. ranking articles by similarity score to produce final results. The goal of this system is to produce an overall more

intuitive and effective search function while maintaining search performance across vast knowledge bases like Wikipedia.

2 Related Work

2.1 Information Retrieval Systems

Traditional information retrieval systems have relied on lexical matching approaches, where documents are ranked based on the presence and frequency of query terms. The vector space model, introduced in the 1970s, represents both documents and queries as vectors in a high-dimensional term space, enabling similarity calculations through geometric operations. TF-IDF weighting emerged as a fundamental technique in this framework, combining term frequency within documents with inverse document frequency across the collection to identify distinctive and informative terms.

2.2 Wikipedia Search and Knowledge Base Retrieval

Wikipedia has been explored as a valuable lexical semantic knowledge base with high potential in diverse Natural Language Processing tasks. Zesch et al. (2008) [4] developed APIs for programmatic access to Wikipedia and Wiktionary, enabling large-scale NLP applications including semantic relatedness computation, text categorization, and information retrieval. Their work highlighted that Wikipedia contains rich lexical semantic information including knowledge about named entities, domain-specific terms, and current developments that are rarely available in traditional linguistic knowledge bases. While our work treats Wikipedia as a collection of text documents for vector retrieval, Zesch et al. [4] show its potential as a structured semantic knowledge base. Their approach highlights that Wikipedia inherently encodes meaning through links, categories, and definitions, features we could incorporate into our evaluation or future system extensions to improve context awareness.

2.3 TF-IDF and Vectorization Approaches

Singh and Shashi (2019) [3] investigated the effectiveness of various text vectorization methods for identifying unifiable news articles, comparing bag-of-words representations with TF-IDF scores, word embeddings, and document embeddings for text clustering tasks. Their framework applied text clustering to news articles related to trending topics

using k-means algorithm, with experiments on the DUC 2004 benchmark dataset indicating that TF-IDF vectorization was the most effective method, producing clusters with higher purity compared to Word2Vec and Doc2Vec approaches. This finding is particularly relevant to our work, as it demonstrates that despite the emergence of neural embedding methods, TF-IDF remains competitive for document similarity tasks, especially when computational efficiency is a concern. TF-IDF's superiority has been demonstrated in several studies for text classification, with the method producing more appropriate clusters than machine learning-based word embedding models.

2.4 Semantic Search Engine Optimization

Pan (2020) [2] addressed the optimization of information retrieval algorithms for digital libraries based on semantic search engines. The work identified limitations in existing semantic search engine ranking approaches, noting that they were one-sided in their sorting approach and had difficulty determining coefficients in ranking formulas. Pan's approach used genetic algorithms to optimize these coefficients, improving search result ranking. While our work focuses on Wikipedia rather than digital libraries, and uses TF-IDF rather than ontology-based semantic search, both approaches share the goal of moving beyond simple keyword matching to capture semantic relationships between queries and documents.

2.5 Evaluation Dataset

Feddoul and Schindler [1] developed KeySearchWiki, an automatically generated dataset for keyword search over Wikidata, containing over 16,000 queries and their relevant results. The dataset is based on Wikidata and Wikipedia set categories which are refined and combined to derive more complex queries, including multi-keyword queries. KeySearchWiki is particularly designed for the Type Search task, where the goal is to retrieve a list of entities having a specific type given by a user query. The dataset was automatically generated by leveraging Wikipedia set categories as data sources for both relevant entities and queries, with relevant entities gathered by navigating the Wikipedia category hierarchy across all available languages. This provides a robust evaluation benchmark that addresses the scarcity of manually-created evaluation datasets for knowledge graph search, enabling systematic assessment of search quality across diverse

query types.

3 Data

3.1 Wikipedia Corpus

We constructed our document collection from the English Wikipedia XML dumps provided by the Wikimedia Foundation. Wikipedia distributes its content in multistream files, which partition the complete encyclopedia into manageable segments. We processed these multistream files in batches to extract the article contents.

The raw Wikipedia dumps contain the complete text and metadata for millions of articles in MediaWiki XML format. Each article includes the page title, full text content with wiki markup, revision history, and associated metadata such as categories and interlanguage links. For our corpus, we extract article titles, the first subsection of text, and the category data from each document.

Our full extraction yields approximately 1.9 million English Wikipedia articles. To ensure computational feasibility during development and to align with our evaluation dataset, we filtered the corpus to include only articles that correspond to entities referenced in the KeySearchWiki dataset. This filtering ensures that relevant articles exist in our collection for evaluation while maintaining a corpus of several tens of thousands of articles with diverse topics and structures. The filtered subset represents a realistic search scenario with sufficient scale to demonstrate the effectiveness of our approach.

3.2 KeySearchWiki Dataset

For systematic evaluation and filtering for our search system, we used KeySearchWiki, a publicly available benchmark dataset specifically designed for evaluating keyword search over knowledge graphs. While KeySearchWiki was originally developed for Wikidata entity retrieval, we adapt it for Wikipedia article search specifically, limiting the scope.

KeySearchWiki contains 16,605 queries with corresponding relevant entities. The queries are expressed in natural language, making them representative of real user information needs rather than structured or formal query syntax. For each query, KeySearchWiki provides a list of relevant entities identified by their Wikidata IRIs (Internationalized Resource Identifiers). These relevance judgments were generated by navigating the Wikipedia cate-

gory hierarchy across all available language editions to gather comprehensive entity sets. The dataset follows standard TREC (Text REtrieval Conference) format, with query files containing query identifiers and text, and qrels files specifying document relevance.

Since KeySearchWiki was designed for Wikidata entities rather than Wikipedia articles, we map between the two resources using the established correspondence between Wikidata entities and Wikipedia articles. We filtered our Wikipedia corpus to prioritize articles that appear in KeySearchWiki relevance judgments, ensuring our evaluation has appropriate coverage. This adaptation allows us to use KeySearchWiki’s comprehensive relevance judgments while searching over Wikipedia article text.

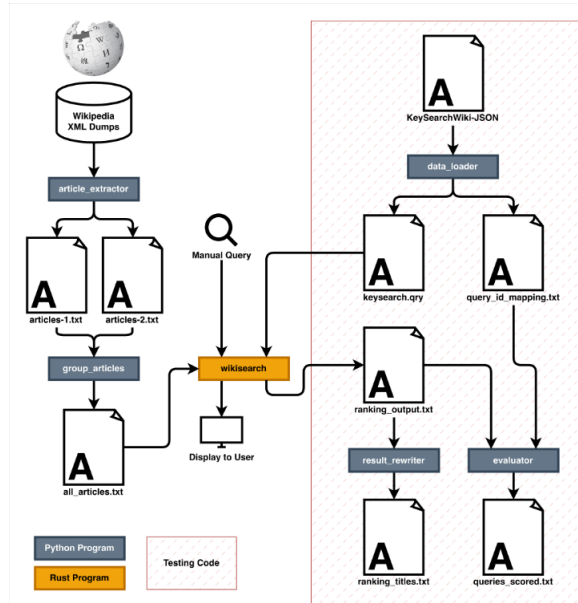
KeySearchWiki is freely available under the Creative Commons Attribution 4.0 International License and can be accessed through the project’s GitHub repository and Zenodo archive.

Table 1: KeySearch Dataset Properties

Property	Description	Example
queryID	Unique identifier of the query: prefix-number, where prefix is NT, MK, or MH.	NT1149 MK79540 MH161
query	Natural-language query of the form: <keyword1 keyword2 ... target>	male television actor human
keywords	Wikidata IRIs or literals for each keyword, including label, types, and boolean isiri.	{ "iri": "Q10798782", "label": "television actor", "isiri": true, "types": [{"type": "Q28640", "typeLabel": "profession"}]}
target	Target entity relevant to the query.	{ "iri": "Q5", "label": "human"} }
relevantEntities	Entities marked as relevant for the query.	{ "iri": "Q16904614", "label": "Zoological Garden of Monaco"} }

4 Methodology

4.1 System Architecture



To build our system, we split each functionality into an independent program, which can be grouped into three main purposes: data preparation, the search engine, and evaluation. All programs besides the search engine are written in Python. The search engine, called `wikisearch`, was written in Rust, which is a programming language that compiles directly to machine code. The decision to use Rust was made to increase performance when running TF-IDF calculations on large amounts of data, which Python, an interpreted language, proved too slow to handle.

The first group of programs handles preparing the Wikipedia article corpus into a format that the search engine can read. The main inputs of this process are the Wikipedia XML dumps downloaded from the Wikimedia foundation. These files are distributed as bzip2 files, which we extracted to obtain the raw XML format. Our `article_extractor` program then reads each of the XML files to process the data. The XML structure of each file is first parsed using the ElementTree XML API in the Python standard library into a tree structure. For each article, the title and full text are extracted using XPath syntax.

Once the title is obtained, we filter out articles so that only articles that appear as relevant entities in the KeySearchWiki data set are available to the search engine. However, the titles in the KeySearchWiki dataset represent Wikidata items, which do not necessarily match the Wikipedia ar-

ticle title obtained from the dumps. For example, a Wikipedia article for the 2015 Adele song is titled "River Lea (song)" while the Wikidata item is titled just "River Lea", since Wikidata titles are not unique. Therefore, we used a dump of all Wikidata items to obtain a mapping between Wikidata entity ids included in the KeySearchWiki dataset (such as Q22682112) and Wikipedia article titles. This mapping is used to correctly filter relevant Wikipedia articles.

After filtering articles, each article's text is transformed from the Wikitext format to raw text. We used the `mwparserfromhell` library to strip Wikitext code, including links, infoboxes, citations, and other metadata. We truncated the articles to only include the text from the lead section, before any subsections. The final step for preparing the articles was combining files produced by each batch of XML dumps into one large file using the `group_articles` program.

The core search engine itself (`wikisearch`), written in Rust, has two modes of operation. The interactive mode allows the user to manually enter queries through the terminal, and prints the results to the user. This is primarily for qualitative testing and sanity checks. The second mode runs a batch of queries from a query file and outputs a file containing the results of all queries in the batch.

The testing and evaluation side of the system involves the `data_loader`, which parses the KeySearchWiki dataset to produce a file with all queries for the `wikisearch` engine and a mapping of query IDs to query text. In addition, the `evaluator` program uses the ranked output of `wikisearch` and the query ID mapping file to calculate the precision, recall, and F1 scores for each query, as well as the mean average precision and recall. Finally, the `result_rewriter` program converts the ranked output into human readable query text and article titles for manual verification and qualitative analysis.

4.2 Text Preprocessing

Before vectorization, we preprocessed both queries and documents to ensure consistency. Our preprocessing pipeline converts raw text into a normalized sequence of stemmed terms suitable for TF-IDF calculation.

We tokenized text by splitting on whitespace, treating each section as a token. This simple tokenization approach is sufficient for English text and aligns with the inherent structure of our Wikipedia corpus, where sentences have already been seg-

mented and formatted during the extraction process.

Punctuation marks are handled in two ways. First, standalone punctuation tokens (e.g., periods, commas, colons, parentheses, quotation marks) are removed entirely as they carry no semantic content for retrieval. Second, punctuation embedded within words is stripped to isolate the alphabetic content. For example, "U.S.A." becomes "USA" and "can't" becomes "cant".

We remove a comprehensive list of stopwords that occur commonly across documents but contribute little to semantic meaning. Our stopword list includes articles (a, an, the), conjunctions (and, or, but), prepositions (in, on, at, by, for, from, to, with), auxiliary verbs (is, am, are, was, were, be, been, being, have, has, had), modal verbs (can, could, may, might, must, shall, should, will, would), and pronouns (I, you, he, she, it, we, they, me, my, his, her, its, their). This list totals approximately 100 stopwords. This reduces index size and computation time while focusing the vocabulary on content-bearing terms.

All text is converted to lowercase to eliminate case distinctions. This ensures that "Climate", "climate", and "CLIMATE" are treated as identical terms, improving recall by combining case variants.

We apply the Porter Stemmer algorithm to reduce words to their morphological root forms. Stemming collapses variants into a common stem, enabling queries to match documents regardless of specific word forms. For example, "running", "runs", "ran", and "runner" all reduce to the stem "run". Stemming is critical for our system because Wikipedia articles may use different word forms than user queries. A search for "political philosophers" should retrieve articles about "political philosophy" or discussing individual "philosophers", even though the exact query terms may not appear. Stemming enables this matching by normalizing all variants to shared stems.

During Wikipedia article extraction, we identify category tags embedded in the article source (formatted as [[Category:category name]]). Categories provide explicit semantic classification—for example, an article about anarchism might include categories like "Political philosophy", "Social theories", and "Anti-capitalism". These categories often contain terms highly relevant to search queries but which may not appear in the article body itself.

We extract category names and integrate them

into the article text by appending the category terms to the end of the processed content. This strategy treats categories as additional document content, increasing the likelihood that relevant articles will match category-related queries. For instance, a query for "political philosophy movements" gains additional matching terms from articles tagged with the "Political philosophy" category, even if those exact words appear infrequently in the article introduction.

After preprocessing, each document and query is represented as a sequence of lowercase, stemmed terms with stopwords and punctuation removed. This normalized representation forms the input to our TF-IDF vectorization process. The preprocessing pipeline substantially reduces vocabulary size—from hundreds of thousands of distinct word forms in the raw text to tens of thousands of unique stems—while preserving the semantic content necessary for effective retrieval.

The final articles file is in the following format: ".I" followed by a sequential article ID on the same line, ".T" followed by the article title on the next line, and ".W" followed by the article text in paragraphs. The processed query file is in the same format minus the ".T" line, since queries do not have titles.

4.3 TF-IDF Calculation

The TF score, or term frequency, is calculated for each query or article by simply counting the number of instances of each word in that query or article. We normalize this by dividing by the number of words.

The IDF scores, or inverse document frequency, of each term are calculated by dividing the number of documents in the corpus by the number of documents that contain that given term, followed by taking the natural logarithm of the result.

4.4 Cosine Similarity

We use cosine similarity to calculate the relevance of a query to an article in the corpus. Given a query and article vector of TF-IDF values for each word, we find the dot product between the two vectors and divide the result by the norm of both vectors multiplied.

5 Results and Evaluation

Using all 1,960,000 filtered articles, we obtained an average precision of 0.1242 and average recall

of 0.07537. While these values appear low, they are largely a consequence of evaluation constraints rather than poor retrieval behavior.

Despite these limitations, qualitative analysis indicates that the system consistently retrieves semantically coherent and relevant articles, particularly for multi-term descriptive queries. For example, the query {"art engineer urban planner scientist human"} returns prominent urban planners and architects such as Ildefons Cerdà, Le Corbusier, and Robert Moses, demonstrating effective conceptual matching. Many of these results would be difficult to retrieve using lexical search alone.

Overall, these findings show that our methodology provides a strong and computationally efficient baseline for semantic search over Wikipedia. Although standard metrics against KeySearchWiki underestimate perceived retrieval quality in this setting, qualitative evidence confirms that the system accurately captures meaningful semantic relationships between queries and articles.

6 Limitations and Challenges

Despite producing a functional baseline for semantic search over Wikipedia, our system faces several limitations arising from computational constraints, dataset design, and characteristics of natural language queries.

6.1 Scalability Constraints

Processing the full English Wikipedia corpus, which contains over five million articles, posed significant computational challenges. Constructing TF-IDF vectors at this scale requires efficient memory management, fast disk I/O, and optimized numerical operations. Our initial Python prototypes were too slow for full-corpus processing, requiring us to re-implement the vectorization in Rust. While this substantially improved the speed, scalability remains a core challenge. Even with optimizations such as multithreading, repeated full-corpus vectorization is time-consuming, and large-scale semantic search poses continual challenges.

6.2 Short Query Limitations

A substantial subset of KeySearchWiki queries, as well as manually input queries, are short natural-language descriptions consisting of only 2–3 words (e.g., “film composer,” “British poet”). These short queries consistently produced lower similarity scores and poorer retrieval quality. TF-IDF

inherently performs better when queries contain multiple descriptive terms, allowing the vectorizer to capture more discriminative signal. With very short queries, most high-IDF terms are absent, and the resulting vectors are too sparse to effectively differentiate among thousands of candidate articles.

To mitigate this, we introduced sublinear TF scaling to reduce the dominance of frequent terms and incorporated Wikipedia category names directly into article text during preprocessing. These strategies helped improve recall for category-driven queries, but short queries remain a difficult edge case for TF-IDF-based systems.

6.3 Mismatch Between KeySearchWiki and Our Corpus

KeySearchWiki was designed for Wikidata entity retrieval, not Wikipedia article retrieval. Although most entities in KeySearchWiki correspond to Wikipedia articles, the mapping is not complete. Some queries reference entities that exist only in Wikidata or whose associated Wikipedia article was absent from our filtered corpus. As a result, certain queries in our evaluation set had no relevant documents in our index, which meant the evaluation system skipped these queries.

6.4 Evaluation Challenges

The KeySearchWiki dataset includes many queries with over 100 relevant articles, while our system only outputs the top 10 articles. This hurts the recall score for these queries, since our system can at maximum only score a recall of 0.1 if there are 100 articles in the expected output. In addition, some queries in the dataset have only a couple relevant articles, which hurts accuracy because our system always outputs 10 results.

Future evaluation could be improved if our system could generate more or less results per query based on some TF-IDF confidence threshold. Until then, evaluation results must be interpreted with caution, and qualitative analysis remains a primary tool for assessing system behavior.

7 Conclusion

In this project, we developed a vectorized semantic search engine for Wikipedia using TF-IDF representations and cosine similarity as a baseline for conceptual retrieval over large text corpora. Motivated by the limitations of Wikipedia’s default search, which relies primarily on exact matching,

we designed a system capable of capturing the underlying meaning of user queries, even when terminology diverges from the phrasing used in article titles.

Our system combines a preprocessing pipeline, TF-IDF vectorization, and similarity-based ranking to retrieve relevant articles from a large subset of the English Wikipedia corpus. Using the KeySearchWiki dataset and qualitative evaluation, we demonstrated that TF-IDF remains a competitive approach for semantic retrieval, particularly when combined with stemming, stopword removal, and category augmentation. The system successfully retrieves conceptually related content and provides a strong foundation for more advanced methods.

At the same time, our experiments highlight the challenges inherent in scaling semantic search to a corpus exceeding one million documents. Short queries, incomplete alignment between KeySearchWiki and Wikipedia, and evaluation constraints all reveal the limitations of this project. Nevertheless, our optimized Rust implementation and integration of category information show that classical IR methods can be adapted to operate effectively at Wikipedia scale.

Looking forward, this work establishes a practical and extensible baseline for semantic search over large knowledge resources. As we expand our system to include Wikidata entities and refine our evaluation framework, we aim to develop a more comprehensive and semantically informed search engine capable of connecting user language to the knowledge-base of Wikipedia and Wikidata.

References

- [1] Leila Feddoul and S. Schindler. fusion-jena/keysearchwiki v1.2.2. Technical report, Friedrich Schiller University Jena, 2023.
- [2] Z. Pan. Optimization of information retrieval algorithm for digital library based on semantic search engine. *2020 International Conference on Computer Engineering and Application (ICCEA)*, 2(1):364–367, 2020.
- [3] Anita Kumari Singh and Mogalla Shashi. Vectorization of text documents for identifying unifiable news articles. *International Journal of Advanced Computer Science and Applications*, 10(7), 2019.
- [4] Torsten Zesch, Christof Müller, and Iryna Gurevych. Extracting lexical semantic knowledge from Wikipedia and Wiktionary. *Proceedings of the Sixth International Conference on Language Resources and Evaluation (LREC’08)*, May 2008.